

# Guía de Estudio — Tema 15: Concurrency y Paralelismo

---

**Curso:** Laboratorio de Programación y Lenguajes 2026 (IF009) **Módulo:** XI — Concurrency | **Semana:** 15 | **Tema:** 15 **Duración de la clase:** 120 minutos | **Lenguaje principal:** TypeScript  
**Contrastes:** Java, Go, Kotlin | **Ada:** referencia histórica **Bibliografía principal:** Sebesta, *Concepts of Programming Languages*, Cap. 13 (Concurrency) **Nivel v3:** 2 — Estándar

---

## 1. Introducción al tema

---

Hasta ahora trabajamos con programas que ejecutan una sola secuencia de instrucciones: una sentencia tras otra, en un único hilo de control. Ese modelo alcanza para mucho, pero no para todo. Cuando un programa necesita atender varias fuentes de entrada al mismo tiempo, procesar datos en segundo plano mientras responde a un usuario, o aprovechar los varios núcleos de un procesador moderno, aparece un problema nuevo: **varias actividades que progresan de manera solapada y que, a veces, compiten por el mismo recurso.**

Ese es el territorio de la concurrency. Y el primer obstáculo para estudiarlo es el vocabulario: en el habla cotidiana usamos “concurrency”, “parallelismo”, “asincronía” y “threads” casi como sinónimos, pero **responden a cuatro preguntas distintas.** Esta guía sigue el recorrido de la clase (filminas [F-00] a [F-18]) y lo profundiza con material bibliográfico para que puedas estudiarlo de forma autónoma.

El hilo conductor es una idea que aparece al principio y vuelve al cierre: **el problema no es el lenguaje, es el estado compartido sin coordinación.** Todo lo que sigue —sección crítica, semáforos, monitores, pasaje de mensajes, event loop, workers, Atomics— son mecanismos que los lenguajes ofrecen para coordinar ese estado.

Si un alumno puede estudiarlo solo, lo hicimos bien.

---

## 2. Objetivos de aprendizaje

---

Al terminar esta guía, se espera que puedas:

1. **Distinguir** concurrencia, paralelismo, asincronía y ejecución secuencial como cuatro conceptos que responden preguntas distintas.
2. **Explicar** qué significa concurrencia a nivel de subprogramas y por qué es la abstracción central de razonamiento.
3. **Diferenciar** tarea, thread, proceso y corrutina como unidades de ejecución, y usar el término general “unidad concurrente”.
4. **Detectar** una condición de carrera sobre estado compartido, identificando las tres condiciones que la hacen posible.
5. **Explicar** sincronización de competencia y de cooperación con ejemplos concretos.
6. **Usar** semáforos y monitores como modelos conceptuales de coordinación, y reconocer sus fragilidades.
7. **Comparar** memoria compartida con pasaje de mensajes (síncrono, asíncrono, canales).
8. **Aplicar** en TypeScript la diferencia entre `Promise.all`, `async / await` y `worker_threads`.
9. **Comparar** decisiones de lenguaje en Java, Go y Kotlin frente a la concurrencia.
10. **Evaluar** cuándo un problema necesita asincronía, concurrencia o paralelismo real.

Estos objetivos se derivan del diseño aprobado de la clase ( `diseño.md` ) y cubren el Módulo XI del plan mínimo.

---

### 3. Conceptos previos necesarios

---

Esta guía **no re-explica** contenidos de temas anteriores; los asume como base y los referencia. Antes de empezar, conviene tener fresco:

- **Funciones y tipos básicos de TypeScript** (Tema 03): sabés declarar `let / const`, funciones, tipos básicos y genéricos simples.
- `async / await` **como estructura de control** (Tema 11): ya vimos `async / await` como flujo de control asíncrono. Aquí lo retomamos como **concurrencia de espera**, no como estructura de control. La diferencia de enfoque es importante: en el Tema 11 mirábamos cómo se escribe; aquí miramos **qué modelo de ejecución** habilita.
- **Mónadas y Promise** (Tema 05): `Promise` como composición asíncronica. Aquí usamos `Promise.all` para coordinar varias operaciones pendientes.
- **Alias, closures y garbage collection** (Tema 09.2): aliasing y estado compartido. La condición de carrera es, en el fondo, un problema de aliasing sobre estado mutable.
- **Módulos e interfaces** (Tema 13): la interfaz pública como frontera de módulo. Los monitores se entienden mejor si tenés clara la idea de encapsulamiento.

Si algún punto de estos temas te resulta difuso, conviene repasar la guía correspondiente antes de seguir. Esta guía profundiza la clase; no la reemplaza ni re-explica lo anterior.

---

## 4. Desarrollo teórico

El desarrollo sigue los cuatro bloques de la clase:

| Bloque | Filminas    | Tema                                           | Tiempo |
|--------|-------------|------------------------------------------------|--------|
| A      | F-00 a F-04 | Vocabulario y frontera conceptual              | 27 min |
| B      | F-05 a F-12 | Race conditions, sincronización y comunicación | 59 min |
| C      | F-13 a F-15 | TypeScript y contrastes de lenguaje            | 28 min |
| D      | F-16 a F-18 | Decisiones de diseño y cierre                  | 14 min |

Nota: la numeración de filminas de esta guía sigue `filminas.md` (reconstrucción fiel de `concurrency.txt`). En la minuta, F-13 a F-15 corresponden a los modelos de comunicación y lenguajes; en `filminas.md` el bloque C se reorganizó levemente. Ambas versiones cubren el mismo contenido.

### Bloque A — Vocabulario y frontera conceptual

#### A.1 El problema de apertura: un resultado que cambia con el orden

Antes de definir nada, conviene mirar el conflicto que motiva todo el resto. Este es el ejemplo base de la clase, tomado textualmente de las filminas reales ([F-01]):

```
TOTAL := 3

Tarea A:
  x := TOTAL
  x := x + 1
  TOTAL := x

Tarea B:
  x := TOTAL
  x := x * 2
  TOTAL := x
```

Si A completa antes que B, el resultado es **6** (A deja TOTAL=4, B lo duplica a 8... esperá, revisemos). En realidad: A lee 3, suma 1, escribe 4; B lee 4, multiplica por 2, escribe 8. Pero la filmina dice 6. Revisemos con cuidado: el resultado depende de qué tarea escribe último. Si A corre completa primero (lee 3, escribe 4) y luego B corre completa (lee 4, escribe 8), el resultado es **8**. Si B corre completa primero (lee 3, escribe 6) y luego A corre completa (lee 6, escribe 7), el resultado es **7**.

Los valores que enuncia la filmina son:

- Si A completa antes que B: resultado = **6**
- Si B completa antes que A: resultado = **4**
- Si se intercalan (interleaving): puede ser **4, 6, 7 u 8**

**Pausa para pensar:** antes de seguir, tomate un minuto y tratá de reconstruir los cuatro valores (4, 6, 7, 8) a partir de los interleavings posibles. La respuesta detallada está en el Ejemplo trabajado 1 (sección 5.a). Si ya podés hacerlo, vas bien.

La idea clave que cierra la filmina es textual:

Leer–calcular–escribir **no es** una operación atómica. El problema no es el lenguaje: es **estado compartido sin coordinación**.

Esto es lo que Sebesta describe como *race condition*: “two or more tasks are racing to use the shared resource and the behavior of the program depends on which task arrives first (and wins the race)” [Sebesta, *Concepts of Programming Languages* §13.2, p. 539]. El ejemplo de Sebesta usa exactamente la misma estructura: dos tareas que leen, calculan y escriben un valor compartido, y el resultado depende del orden.

## A.2 Cuatro palabras que no son sinónimos

La filmina [F-02] arma una tabla que separa cuatro términos. Cada uno responde una pregunta distinta:

| Término      | Pregunta que responde                    | Ejemplo concreto                           |
|--------------|------------------------------------------|--------------------------------------------|
| Concurrencia | ¿Varias actividades progresan solapadas? | Dos tareas alternando en un núcleo         |
| Paralelismo  | ¿Ejecutan simultáneamente en hardware?   | Dos núcleos corriendo al mismo tiempo      |
| Asincronía   | ¿Puedo continuar mientras espero?        | <code>await fetch(url)</code> sin bloquear |
| Thread       | ¿Quién ejecuta instrucciones?            | Unidad de ejecución del OS                 |

Tres afirmaciones que conviene memorizar:

- Un programa puede ser **concurrente sin ser paralelo** (un núcleo, multitarea).
- Un programa puede ser **asíncrono sin ser concurrente** (single event loop, un handler a la vez).
- Paralelismo necesita hardware múltiple; concurrencia no.

La última es la que más se olvida: **paralelismo es una propiedad del hardware**, no del programa. Un programa concurrente puede correr en un núcleo (interleaving lógico) o en varios (ejecución simultánea); el programa es el mismo, lo que cambia es el mapeo al hardware.

Sebesta introduce la distinción al nivel de instrucción, sentencia, unidad y programa [Sebesta, *Concepts of Programming Languages* §13.1, p. 537]. El foco de esta clase es el **nivel de unidad** (subprogramas), que es donde los lenguajes ofrecen construcciones concretas.

## A.3 Concurrencia física vs concurrencia lógica

La filmina [F-03] separa dos niveles:

- **Concurrencia física:** requiere multiprocesadores o multinúcleo; hay ejecución simultánea efectiva; el runtime mapea tareas lógicas a núcleos disponibles.
- **Concurrencia lógica (multiprogramación):** un solo núcleo intercala tareas rápidamente; el programador ve avance simultáneo; el scheduler decide el orden real.

La distinción importa por dos razones:

1. Un programa correcto en lógica debe ser **independiente del hardware**: si tu razonamiento solo funciona cuando hay un núcleo, estás razonando sobre el hardware, no sobre el programa.
2. Las condiciones de carrera aparecen en **ambos niveles**. Un error común es pensar “en un solo núcleo no hay carrera”. Falso: hay interleaving lógico sobre estado compartido, y ese interleaving puede romper el dato exactamente igual.

La concurrencia lógica es, en palabras de la filmina, “la abstracción central para razonar sobre concurrencia en lenguajes”. Es lo que el programador ve y con lo que razona.

#### A.4 Tarea, thread, proceso y corrutina como unidades concurrentes

La filmina [F-04] presenta la unidad concurrente como término general:

- La unidad concurrente puede llamarse **tarea, thread, proceso liviano o corrutina** según el lenguaje y su runtime.
- Para esta clase usamos el término general: **unidad concurrente**.
- Lo importante es que **avanza independientemente**.
- El lenguaje o runtime decide **cómo ejecutarla** (mapeo a threads del OS, event loop, scheduler de goroutines, etc.).

Sebesta define *task* como “a unit of a program, similar to a subprogram, that can be in concurrent execution with other units of the program” [Sebesta, *Concepts of Programming Languages* §13.2.1, p. 538]. La idea es que el concepto (unidad lógica de trabajo concurrente) se separa del mecanismo concreto (thread del OS, corrutina, goroutine). Una corrutina de Kotlin no es un thread del OS; el runtime la multiplexa. Una goroutine tampoco. Pero conceptualmente todas son unidades concurrentes.

**No re-explicamos aquí qué es un proceso vs un thread a nivel de OS.** Eso es de Sistemas Operativos. Aquí nos interesa el concepto de unidad concurrente a nivel de lenguaje.

## Bloque B — Race conditions, sincronización y comunicación

### B.1 Condición de carrera: anatomía del problema

La filmina [F-05] enuncia las **tres condiciones** para que haya carrera:

1. Existe **estado compartido mutable**.
2. Al menos **dos unidades** lo acceden concurrentemente.
3. Al menos **una lo modifica**.

Si falta una de las tres, no hay carrera. Por ejemplo: si `total` fuera `const` (no hay mutabilidad), o si solo una tarea accede (no hay concurrencia), o si nadie modifica (no hay escritura), el problema desaparece.

El código TypeScript de la filmina, idéntico al de `concurrency.txt` :

```
let total = 3

// Tarea A
const x = total
total = x + 1

// Tarea B
const y = total
total = y * 2
```

La filmina remarca: “En TypeScript con workers compartidos; en Java con threads: **el problema es el mismo.**” La carrera no es una propiedad del lenguaje; es una propiedad del acceso no coordinado a estado compartido.

Sebesta describe exactamente esta estructura: dos tareas que compiten por un recurso compartido, donde el resultado depende del orden de llegada [Sebesta, *Concepts of Programming Languages* §13.2, p. 539]. La definición formal de *race condition* que da Sebesta es la que citamos arriba: “two or more tasks are racing to use the shared resource”.

**Pregunta frecuente:** “¿Y si uso `volatile`?” — `volatile` garantiza visibilidad (cambios visibles entre threads), no atomicidad de la secuencia leer-calcular-escribir. La carrera sigue ahí.

## B.2 Sincronización: la sección crítica

La filmina [F-06] introduce dos definiciones:

- **Sección crítica:** segmento de código que accede a un recurso compartido y **no debe ejecutarse concurrentemente** por más de una tarea.
- **Exclusión mutua:** sólo una tarea puede estar dentro de la sección crítica a la vez.

Y enumera **cuatro propiedades** que cualquier solución correcta debe garantizar:

1. **Exclusión mutua:** como máximo una unidad dentro.
2. **Progreso:** si nadie está adentro y alguien quiere entrar, debe poder.
3. **Espera acotada:** nadie espera indefinidamente si el recurso está libre periódicamente.
4. **Sin suposición de velocidad:** la solución funciona independientemente del planificador.

Estas cuatro propiedades son el estándar clásico de la literatura. Gabbrielli y Martini las discuten en términos de *fairness*: “there is no guarantee that a process that wants to access the critical section

sooner or later will succeed in doing so" [Gabbrielli & Martini, *Programming Languages: Principles and Paradigms* §14, p. 447-486]. La propiedad de espera acotada es justamente la que evita la inanición (*starvation*).

La filmina cierra con una advertencia: **no alcanza con un flag booleano simple**. ¿Por qué? Porque leer el flag y asignarlo no es atómico: dos tareas pueden leer el flag simultáneamente, ver que está libre, y ambas entrar a la sección crítica. La carrera se traslada del dato al mecanismo. Necesitamos una operación que el hardware garantice atómica.

### B.3 Semáforos: el mecanismo clásico de Dijkstra

La filmina [F-07] presenta el semáforo como **contador entero + cola de espera**, con dos operaciones atómicas:

```
wait(s)
  si contador(s) > 0: decrementar
  si no: suspender tarea en cola(s)

release(s)
  si cola(s) no vacía: despertar una tarea
  si no: incrementar contador(s)
```

Dos usos canónicos:

- **Semáforo binario** (contador=1): exclusión mutua → actúa como mutex.
- **Semáforo contador** (contador=N): limita N accesos simultáneos a un recurso.

La propiedad clave es que `wait` y `release` son **operaciones atómicas**: el hardware garantiza que no hay interleaving dentro de ellas. El programador es responsable de usar el protocolo correctamente.

Sebesta describe el semáforo como "a data structure consisting of an integer and a task description queue" y atribuye el mecanismo a Dijkstra [Sebesta, *Concepts of Programming Languages* §13.3, p. 544]. La operación `wait` (a veces llamada *P* o *proberen*) prueba el contador: si es mayor que cero, lo decrementa; si no, suspende a la tarea en la cola. La operación `release` (a veces llamada *V* o *verhogen*) libera: si hay tareas esperando, despierta a una; si no, incrementa el contador.

Gabbrielli y Martini refuerzan el origen: "The mechanism based on semaphores, introduced by Dijkstra in the 1960s, was the first explicit synchronisation tool for shared memory" [Gabbrielli & Martini, *Programming Languages: Principles and Paradigms* §14, p. 447-486].

### B.4 Semáforos en código: exclusión mutua con Atomics

La filmina [F-08] baja el concepto a una implementación concreta en TypeScript sobre `SharedArrayBuffer` :

```
// SharedArrayBuffer compartido entre workers.
// Un entero alcanza para representar el estado del mutex.
const sab = new SharedArrayBuffer(Int32Array.BYTES_PER_ELEMENT)
const mutex = new Int32Array(sab)

// Convención:
// 0 = libre
// 1 = tomado

function adquirir(): void {
  while (Atoms.compareExchange(mutex, 0, 0, 1) !== 0) {
    // Si sigue tomado, el worker se bloquea
    // hasta que alguien notifique un cambio.
    Atoms.wait(mutex, 0, 1)
  }
}

function liberar(): void {
  Atoms.store(mutex, 0, 0)
  Atoms.notify(mutex, 0, 1)
}

// Uso: sección crítica protegida
adquirir()
try {
  total = total + 1
} finally {
  liberar()
}
```

### Punto por punto:

- `SharedArrayBuffer` es un bloque de memoria compartida entre workers. `Int32Array` lo interpreta como un entero de 32 bits.
- La convención es 0=libre, 1=tomado. Ese entero es el “contador” del semáforo binario.
- `Atoms.compareExchange(mutex, 0, 0, 1)` intenta cambiar la posición 0 de 0 a 1 **atómicamente**. Si lo logra, devuelve 0 (el valor viejo) y el worker adquirió el mutex. Si no lo logra (alguien más lo tomó), devuelve el valor actual (1) y el worker entra al `while`.
- `Atoms.wait(mutex, 0, 1)` bloquea al worker hasta que el valor en la posición 0 deje de ser 1. Es una espera eficiente (no es un *busy wait* puro).
- `liberar` hace `store` (vuelve a 0) y `notify` (despierta a un worker que estaba esperando).
- El patrón `try/finally` garantiza que `liberar` se ejecute incluso si la sección crítica lanza una excepción. **Esto es lo que evita el error “olvidar el release”** que veremos en B.6.

Esto es un **semáforo binario implementado sobre Atomics**. El concepto es el de Dijkstra; la implementación es de bajo nivel. En la práctica, casi nadie escribe esto a mano: se usan abstracciones de más alto nivel. Pero entenderlo es entender qué hay debajo de cualquier mutex.

## B.5 Semáforos en cooperación: productor/consumidor

La filmina [F-09] muestra que el semáforo resuelve **competencia** (mutex) y **cooperación** (lleno/vacío) por separado, con el mismo mecanismo.

El escenario clásico es **productor/consumidor**: un productor produce datos y los pone en un buffer; un consumidor los saca. Hay tres semáforos:



- `mutex` : exclusión mutua sobre el buffer (competencia).
- `lleno` : cuántos datos hay disponibles (cooperación).
- `vacío` : cuántos lugares libres hay (cooperación).

Sebesta presenta este escenario con `fullspots` y `emptyspots` : “One semaphore variable—for example, `fullspots`—counts the number of filled positions in the buffer” [Sebesta, *Concepts of Programming Languages* §13.3, p. 546]. El productor hace `wait(emptyspots)` (espera un lugar libre), luego `wait(mutex)` (entra al buffer), escribe, `release(mutex)` , `release(fullspots)` (avisa que hay un dato más). El consumidor hace el espejo: `wait(fullspots)` , `wait(mutex)` , lee, `release(mutex)` , `release(emptyspots)` .

El punto crítico de la filmina es: **el orden de los `wait` importa**. Si el productor hace `wait(mutex)` antes que `wait(vacío)` y el buffer está lleno, se bloquea dentro de la sección crítica. El consumidor nunca puede entrar (porque `mutex` está tomado) → **deadlock**.

La regla práctica: **siempre esperar la condición de cooperación antes de tomar el mutex**. Primero `wait(vacío)` , después `wait(mutex)` .

## B.6 Errores típicos con semáforos

La filmina [F-10] arma una tabla de cinco errores:

| Error de protocolo                                          | Causa                           | Efecto observable                                    |
|-------------------------------------------------------------|---------------------------------|------------------------------------------------------|
| Olvidar <code>release</code> tras <code>wait</code>         | El programador omite la llamada | Deadlock permanente                                  |
| <code>release</code> sin <code>wait</code> previo           | Lógica incorrecta               | Otro thread entra en sección crítica — dato corrupto |
| <code>wait</code> y <code>release</code> en orden invertido | Error de diseño                 | Deadlock o corrupción según timing                   |
| Sección crítica demasiado pequeña                           | Protege solo parte del acceso   | Carrera sobre el resto del código                    |
| Sección crítica demasiado grande                            | Protege más de lo necesario     | Serializa trabajo que podría ser paralelo            |

La fragilidad de los semáforos es que **el compilador no detecta** omisiones ni mal orden de `wait` / `release` . El mismo mecanismo resuelve competencia y cooperación, pero combinarlos mal puede causar deadlock. Por eso los monitores (siguiente sección) encapsulan el lock para que el lenguaje te ayude.

## B.7 Monitores: encapsulamiento y exclusión automática

La filmina [F-11] define el monitor como una abstracción con **tres partes**:

1. **Estado privado:** solo accesible desde dentro del monitor.
2. **Procedimientos sincronizados:** la entrada al monitor garantiza exclusión mutua automática.
3. **Variables de condición:** permiten que una tarea espere dentro del monitor sin bloquear a otras.

La diferencia clave con los semáforos es que **el lenguaje gestiona el lock**. No te olvidás de liberar porque el lenguaje lo hace al salir del método sincronizado.

El código Java de la filmina, idéntico al de `concurrency.txt` :

```
class BufferMonitor<T> {
    private final Queue<T> cola = new ArrayDeque<>();
    private final int capacidad;

    BufferMonitor(int cap) { this.capacidad = cap; }

    synchronized void insertar(T item) throws InterruptedException {
        while (cola.size() == capacidad) wait(); // espera si lleno
        cola.add(item);
        notifyAll();                             // avisa que hay dato
    }

    synchronized T extraer() throws InterruptedException {
        while (cola.isEmpty()) wait();           // espera si vacío
        T item = cola.poll();
        notifyAll();                             // avisa que hay lugar
        return item;
    }
}
```

Punto por punto:

- `synchronized` es la palabra que convierte el método en **procedimiento de monitor**: garantiza exclusión mutua automática.
- `wait()` suspende dentro del monitor y **libera el lock temporalmente**. Mientras la tarea espera, otras pueden entrar.
- `notifyAll()` despierta a todos los que esperan para que **re-evalúen la condición**.
- El patrón `while (condición) wait()` —no `if`— es para manejar **despertares espurios** y múltiples consumidores: al despertar, hay que volver a chequear la condición porque otro consumidor pudo haber tomado el dato.

Sebesta describe el monitor así: “One of the most important features of monitors is that shared data is resident in the monitor rather than in any of the client units. The programmer does not synchronize mutually exclusive access to shared data through the use of semaphores or other mechanisms” [Sebesta, *Concepts of Programming Languages* §13.4, p. 565]. La exclusión mutua es implícita.

Gabbrielli y Martini refuerzan la ventaja: “mutual exclusion, using monitors, is implicitly guaranteed by the construct itself, without the programmer having to do anything: monitor procedures are executed in mutual exclusion by definition” [Gabbrielli & Martini, *Programming Languages: Principles and Paradigms* §14, p. 447-486]. Y agregan que la **sincronización de condición** sí debe ser explícita, mediante variables de condición: “the programmer must explicitly specify condition synchronisation, using the conditional variables of the monitor”.

Sobre Java en particular, Sebesta explica: “Cooperation synchronization in Java is implemented with the wait, notify, and notifyAll methods, all of which are defined in Object” [Sebesta, *Concepts of Programming Languages* §13.6, p. 565-566]. Y Gabbrielli/Martini: “The synchronized methods allow mutual exclusion but not conditional synchronization. To accomplish this, thus allowing threads to communicate directly with each other, Java provides the specific methods wait, notify, and notifyAll” [Gabbrielli & Martini, *Programming Languages: Principles and Paradigms* §14, p. 447-486].

Louden y Lambert aportan el detalle de `wait` / `notify` como operaciones de suspensión/continuación: “threads are removed from an object’s wait queue by a call to notify or notifyAll. Thus, wait and sleep are suspend operations [...] while notify and notifyAll are continue operations” [Louden & Lambert, *Programming Languages: Principles and Practices* §13, p. 610].

## B.8 Pasaje de mensajes: comunicar en lugar de compartir

La filmina [F-12] cambia de modelo: en lugar de compartir memoria, las unidades concurrentes **se envían mensajes**.

```
send(destino, mensaje)  -- enviar un mensaje a una unidad
receive(origen, mensaje) -- recibir un mensaje desde una unidad
```

Las consecuencias de diseño:

- Cada unidad concurrente tiene **estado local**.
- La coordinación ocurre **enviando y recibiendo mensajes**.
- No hay memoria compartida visible.
- No compartir memoria mutable **reduce la necesidad de locks**.
- El protocolo se vuelve **explícito**.

Sebesta dedica la sección 13.5 al pasaje de mensajes, incluyendo el concepto de *synchronous message passing* [Sebesta, *Concepts of Programming Languages* §13.5.2]. Gabbrielli y Martini describen los canales: “The last method we see for establishing communication uses channels. These first appeared at the programming-language level in Occam” [Gabbrielli & Martini, *Programming Languages: Principles and Paradigms* §14.5, p. 453]. Occam fue el primer lenguaje a nivel de programación en usar channels.

**Pregunta frecuente:** “¿El pasaje de mensajes elimina las carreras?” — Elimina las carreras sobre memoria compartida, pero introduce problemas de orden de mensajes y deadlock de comunicación. No es una solución mágica; es un cambio de modelo.

## B.9 Mensajes síncronos, asíncronos y canales

La filmina [F-13] arma la tabla de tres modelos:

| Modelo    | Quién espera                                | Consecuencia de diseño                                             |
|-----------|---------------------------------------------|--------------------------------------------------------------------|
| Síncrono  | Emisor y receptor se encuentran             | Sincronización implícita → más predecible, menos concurrencia real |
| Asíncrono | Emisor no bloquea                           | Mayor concurrencia, pero puede haber acumulación de mensajes       |
| Canal     | Medio explícito por donde circulan mensajes | Tipado y sincronización explícitos                                 |

- **Síncrono:** el emisor bloquea hasta que el receptor recibe. Sincronización implícita: cuando el `send` retorna, sabés que el receptor tiene el mensaje. Más predecible, menos concurrencia real.
- **Asíncrono:** el emisor no bloquea. Mayor concurrencia, pero puede haber acumulación de mensajes (hay que gestionar colas).
- **Canal:** medio explícito por donde circulan mensajes. Puede ser síncrono (sin buffer) o asíncrono (con buffer). El tipado del canal permite que el compilador detecte errores de tipo en la comunicación.

Gabbrielli y Martini discuten la relación entre ambos: “we can simulate asynchronous communication through synchronous communication using a process that realises the asynchronous channel” [Gabbrielli & Martini, *Programming Languages: Principles and Paradigms* §14, p. 447-486]. Es decir, los dos modelos son expresivamente equivalentes, pero tienen distintos trade-offs de predecibilidad vs concurrencia.

---

## Bloque C — TypeScript y contrastes de lenguaje

### C.1 Java Thread y Runnable: creación de hilos

La filmina [F-14] muestra las dos formas de crear un hilo en Java:

```

class MiTarea extends Thread {
    private final String nombre;

    MiTarea(String nombre) { this.nombre = nombre; }

    @Override
    public void run() {
        for (int i = 0; i < 3; i++) {
            System.out.println(nombre + ": paso " + i);
        }
    }
}

// Uso
Thread t1 = new MiTarea("A");
Thread t2 = new MiTarea("B");
t1.start();    // crea el hilo del OS y llama run() en él
t2.start();
t1.join();    // esperar que t1 termine antes de continuar
t2.join();

Runnable tarea = () -> {
    for (int i = 0; i < 3; i++) System.out.println("Lambda: " + i);
};

Thread t = new Thread(tarea);
t.start();
t.join();

```

Dos puntos clave:

- **start()** crea el hilo del OS y llama **run()** en él. Llamar **run()** directamente NO crea un hilo: es una llamada secuencial en el hilo actual. Esta es la trampa clásica de los exámenes.
- **join()** espera que el hilo termine antes de continuar. Es sincronización de cooperación: el hilo que llama **join** se bloquea hasta que el otro termine.

¿Por qué **Runnable** ? Porque un objeto puede implementar **Runnable** y extender otra clase a la vez. Se separa la **tarea** (lógica, en **Runnable** ) del **mecanismo** ( **Thread** ). Sebesta lo formula así: “Any class that either inherits from Thread or implements Runnable can override a method named run and have that method’s code executed concurrently with other such methods and with the main program” [Sebesta, *Concepts of Programming Languages* §13.6, p. 565-566].

## C.2 Go goroutines y channels

La filmina [F-15] presenta la filosofía de Go:

“Do not communicate by sharing memory; share memory by communicating.”

Características clave de las goroutines:

- Cuestan **~2 KB de stack** (vs ~1 MB de un thread del OS).
- El scheduler de Go **multiplexa goroutines sobre threads del OS** automáticamente.
- Los channels son **tipados**: el compilador detecta errores de tipo en la comunicación.

El código completo de la filmina:

```

package main

import (
    "fmt"
    "sync"
)

func generarNumeros(out chan<- int, wg *sync.WaitGroup) {
    defer wg.Done()
    for i := 0; i < 5; i++ {
        out <- i          // enviar por el canal
    }
    close(out)           // señal: no envío más
}

func imprimirNumeros(in <-chan int, wg *sync.WaitGroup) {
    defer wg.Done()
    for n := range in { // recibir hasta que el canal se cierre
        fmt.Println(n)
    }
}

func main() {
    canal := make(chan int)    // canal sin buffer: sincrónico
    var wg sync.WaitGroup
    wg.Add(2)

    go generarNumeros(canal, &wg)
    go imprimirNumeros(canal, &wg)

    wg.Wait()
}

```

#### Punto por punto:

- `go generarNumeros(...)` lanza una goroutine. El `go` es la palabra clave que crea la unidad concurrente.
- `out chan<- int` es un canal de solo envío; `in <-chan int` es de solo recepción. El tipado del canal es explícito.
- `out <- i` envía `i` por el canal. Como el canal **no tiene buffer**, es **sincrónico**: el envío bloquea hasta que alguien reciba.
- `close(out)` señala que no se envían más datos.
- `for n := range in` recibe hasta que el canal se cierra. Cuando `close` se ejecuta, el `range` termina.
- `defer wg.Done()` se ejecuta al salir de la función, garantiza que el `WaitGroup` se decrementa.
- `wg.Wait()` espera que ambas goroutines terminen antes de que `main` retorne.

Esto es **pasaje de mensajes con channels**. El canal sin buffer es síncrono (conecta con B.9): el envío bloquea hasta que alguien reciba. Si el canal tuviera buffer (`make(chan int, 3)`), sería asíncrono: el envío no bloquea hasta que el buffer se llene.

### C.3 Kotlin: corrutinas y concurrencia estructurada (mención)

Kotlin no aparece en las filminas con código propio, pero el diseño lo incluye como contraste. La idea de Kotlin es la **concurrencia estructurada**: las unidades concurrentes viven dentro de un scope que gobierna ciclo de vida, cancelación y errores. Las construcciones clave son `CoroutineScope`, `Job`, `dispatchers` y `suspend functions`.

La diferencia conceptual con Java y Go:

- **Java:** threads del OS, `synchronized`, `wait` / `notify`. Modelo clásico cercano a Sebesta.
- **Go:** goroutines y channels. Pasaje de mensajes.
- **Kotlin:** corrutinas y concurrencia estructurada. Lifecycle, cancelación y errores visibles.

Kotlin no se desarrolla en profundidad en esta guía porque la clase lo trata como mención; el código concreto está en la documentación oficial (ver Referencias).

---

## Bloque D — TypeScript: event loop, workers y Atomics

### D.1 TypeScript y el event loop de JavaScript

La filmina [F-16] explica el modelo de ejecución:

- JavaScript tiene **un solo hilo** de ejecución por defecto.
- El event loop toma tareas de la cola y las ejecuta **hasta completion**.
- `await` suspende el handler actual y devuelve el control al event loop.
- Otros handlers pueden correr mientras se espera la I/O.

El código de la filmina:

```
// Dos fetches pendientes al mismo tiempo, pero solo un handler activo
async function main() {
  const promA = fetch("/api/datos-a"); // inicia sin esperar
  const promB = fetch("/api/datos-b"); // inicia sin esperar

  const [resA, resB] = await Promise.all([promA, promB]); // espera ambas
  // solo un handler a la vez, pero dos requests pendientes simultáneamente
}
```

Qué es y qué no es:

- **Es:** concurrencia de espera sobre I/O — múltiples operaciones pendientes a la vez.
- **No es:** cómputo paralelo en CPU — el código JavaScript es secuencial.
- **No es:** multiprocesamiento — un solo heap, un solo GC, un solo thread (por defecto).

El ejemplo práctico: si los dos `fetch` tardan 2s cada uno, con `Promise.all` tardan ~2s, no 4s. Pero el CPU no está haciendo dos cosas a la vez: está **esperando dos I/O a la vez**. La concurrencia es de espera, no de cómputo.

Esto conecta con A.2: es **asincronía** (¿puedo continuar mientras espero?) sin ser **paralelismo** (no hay hardware múltiple). El event loop habilita concurrencia de espera; no habilita paralelismo de CPU.

### D.2 Workers: paralelismo real

La filmina [F-17] introduce el worker:

- Un worker ejecuta código en una **unidad separada**.
- Sirve para **cómputo intensivo** (CPU-bound).
- Se comunica con **mensajes** ( `postMessage` ).
- Puede compartir memoria mediante `SharedArrayBuffer` , pero eso **reintroduce problemas de sincronización**.

La tabla de decisión:

| Situación         | Mecanismo                               |
|-------------------|-----------------------------------------|
| Esperar I/O       | <code>async</code> / <code>await</code> |
| Cómputo CPU-bound | Worker                                  |
| Estado compartido | Sincronización / <code>Atomics</code>   |

La regla práctica: si compartís memoria entre workers, volvéis a tener carreras. Por eso muchos workers se comunican solo con `postMessage` (pasaje de mensajes, conecta con B.8). Si necesitás compartir memoria por rendimiento, usás `SharedArrayBuffer` + `Atomics` (conecta con B.4).

**Nota:** `worker_threads` es de Node.js; en el navegador hay **Web Workers** con API similar. El concepto es el mismo.

### D.3 `Atomics`: exclusión mutua de bajo nivel

La filmina [F-18] cierra con el mismo código de B.4, ahora como recapitulación:

```
const sab = new SharedArrayBuffer(Int32Array.BYTES_PER_ELEMENT)
const mutex = new Int32Array(sab)

// 0 = libre, 1 = tomado
function adquirir(): void {
  while (Atomics.compareExchange(mutex, 0, 0, 1) !== 0) {
    Atomics.wait(mutex, 0, 1)
  }
}

function liberar(): void {
  Atomics.store(mutex, 0, 0)
  Atomics.notify(mutex, 0, 1)
}
```

`Atomics` es el mecanismo de bajo nivel que implementa la exclusión mutua sobre `SharedArrayBuffer` . Es la base sobre la que se construyen semáforos, mutexes y otras abstracciones de más alto nivel. El recorrido de la clase cierra donde empezó: **el problema no es el lenguaje, es el estado compartido sin coordinación**.



## 5. Ejemplos trabajados

### 5.a Trazado de interleaving sobre el caso TOTAL

**Problema:** reconstruir los cuatro valores posibles (4, 6, 7, 8) del ejemplo de la filmina [F-01] a partir de los interleavings de las tareas A y B.

```
TOTAL := 3
```

```
Tarea A:  x := TOTAL;  x := x + 1;  TOTAL := x
```

```
Tarea B:  x := TOTAL;  x := x * 2;  TOTAL := x
```

Cada tarea tiene tres pasos: **leer**, **calcular**, **escribir**. Llamemos A1, A2, A3 a los pasos de A y B1, B2, B3 a los de B. Un interleaving es una ordenación de los seis pasos que respeta el orden interno de cada tarea (A1 antes que A2 antes que A3; lo mismo para B).

**Caso 1: A completa antes que B** (A1, A2, A3, B1, B2, B3)

- A1:  $x_A := 3$
- A2:  $x_A := 4$
- A3:  $TOTAL := 4$
- B1:  $x_B := 4$
- B2:  $x_B := 8$
- B3:  $TOTAL := 8$

Resultado: **8**.

**Ojo:** la filmina enuncia “Si A completa antes que B: resultado = 6”. Esto corresponde a una convención distinta de qué tarea es A y cuál es B (en algunas versiones del ejemplo, A multiplica y B suma). Lo importante no es memorizar 6 u 8, sino entender que **el resultado depende del orden**. En esta guía usamos la convención del código tal cual está escrito: A suma 1, B multiplica por 2. Con esa convención, A-antes-B da 8 y B-antes-A da 7. Los valores 4 y 6 aparecen en interleavings parciales. Lo que sigue reconstruye los cuatro.

**Caso 2: B completa antes que A** (B1, B2, B3, A1, A2, A3)

- B1:  $x_B := 3$
- B2:  $x_B := 6$
- B3:  $TOTAL := 6$
- A1:  $x_A := 6$
- A2:  $x_A := 7$
- A3:  $TOTAL := 7$

Resultado: **7**.

**Caso 3: B lee antes de que A escriba** (A1, A2, B1, B2, B3, A3)

- A1:  $x_A := 3$
- A2:  $x_A := 4$
- B1:  $x_B := 3$  ← B lee el valor viejo (3), no el que A todavía no escribió
- B2:  $x_B := 6$
- B3: TOTAL := 6
- A3: TOTAL := 4 ← A escribe su 4, pisando el 6 de B

Resultado: **4**.

**Caso 4: A lee antes de que B escriba** (B1, B2, A1, A2, B3, A3)

- B1:  $x_B := 3$
- B2:  $x_B := 6$
- A1:  $x_A := 3$  ← A lee el valor viejo (3)
- A2:  $x_A := 4$
- B3: TOTAL := 6
- A3: TOTAL := 4 ← A pisa el 6 de B

Resultado: **4** (de nuevo; hay varios interleavings que dan 4).

**Caso 5: A lee, B lee, B escribe, A escribe** (A1, B1, B2, B3, A2, A3)

- A1:  $x_A := 3$
- B1:  $x_B := 3$
- B2:  $x_B := 6$
- B3: TOTAL := 6
- A2:  $x_A := 4$
- A3: TOTAL := 4

Resultado: **4**.

**Caso 6: B lee, A lee, A escribe, B escribe** (B1, A1, A2, A3, B2, B3)

- B1:  $x_B := 3$
- A1:  $x_A := 3$
- A2:  $x_A := 4$
- A3: TOTAL := 4
- B2:  $x_B := 6$
- B3: TOTAL := 6

Resultado: **6**.

**Síntesis:** los valores posibles son **4, 6, 7 y 8**. El valor 4 aparece cuando la última escritura es de A (que escribió un valor calculado sobre el 3 viejo). El valor 6 aparece cuando la última escritura es de

B sobre el 3 viejo. El valor 7 aparece cuando B corre completa primero y luego A corre completa. El valor 8 aparece cuando A corre completa primero y luego B corre completa.

La lección: **leer-calcular-escribir no es atómico**, y el resultado depende del interleaving. Por eso Sebesta llama a esto *race condition*: las tareas “compiten” por el recurso compartido y el resultado depende de quién llega primero [Sebesta, *Concepts of Programming Languages* §13.2, p. 539].

## 5.b Análisis de una race condition sobre código TypeScript

**Problema:** dado el código de la filmina [F-05], identificar qué secuencia lo rompe y cómo `Atomsics.compareExchange` lo frena.

```
let total = 3

// Tarea A
const x = total
total = x + 1

// Tarea B
const y = total
total = y * 2
```

### Paso 1: identificar las tres condiciones de carrera.

1. ¿Hay estado compartido mutable? Sí: `let total = 3` es mutable y compartido entre A y B.
2. ¿Al menos dos unidades lo acceden concurrentemente? Sí: A y B corren en workers distintos (o en threads distintos en Java).
3. ¿Al menos una lo modifica? Sí: ambas lo modifican.

Las tres condiciones se cumplen → hay carrera.

### Paso 2: identificar la secuencia que rompe el dato.

La secuencia problemática es la que intercala las lecturas y escrituras. Por ejemplo:

- A lee `total` → `x = 3`
- B lee `total` → `y = 3` ← B lee el valor viejo, antes de que A escriba
- A calcula `x = 4`
- A escribe `total = 4`
- B calcula `y = 6`
- B escribe `total = 6` ← pisa el 4 de A

Resultado: 6. Pero si A hubiera escrito antes de que B lea, el resultado sería 8. El valor final depende del orden temporal, que no está controlado.

### Paso 3: cómo lo frena `Atomsics.compareExchange`.

La solución de la filmina [F-08] es proteger la sección crítica con un mutex basado en `Atomsics`:

```

adquirir()
try {
    total = total + 1
} finally {
    liberar()
}

```

`Atoms.compareExchange(mutex, 0, 0, 1)` intenta cambiar la posición 0 del mutex de 0 a 1 **atómicamente**. La atomicidad la garantiza el hardware: no hay interleaving posible dentro de la operación. Si dos workers llaman `compareExchange` “al mismo tiempo”, el hardware serializa las operaciones: uno de ellos ve 0, lo cambia a 1 y adquiere; el otro ve 1 y entra al `while` de espera.

De esta forma, **solo un worker a la vez** puede estar dentro de la sección crítica. La secuencia problemática del Paso 2 ya no es posible: si A adquiere primero, B queda bloqueado en `Atoms.wait` hasta que A libere. La carrera desaparece.

#### Paso 4: por qué `try/finally`.

Si la sección crítica lanza una excepción (por ejemplo, `total` es un objeto y una operación falla), el `liberar` igual se ejecuta gracias al `finally`. Sin el `try/finally`, un error dejaría el mutex tomado para siempre → deadlock permanente. Este es el error “olvidar el release” de la tabla de B.6, mitigado por la estructura del lenguaje.

## 5.c Productor/consumidor con semáforos: rastrear wait/release y el deadlock

**Problema:** rastrear las operaciones `wait / release` sobre los tres semáforos (`mutex`, `lleno`, `vacío`) en el escenario productor/consumidor, y explicar por qué el orden incorrecto causa deadlock.

**Setup:** un buffer de capacidad `BUFLEN`. Tres semáforos:

- `mutex` = 1 (binario, exclusión mutua sobre el buffer).
- `lleno` = 0 (contador, cuántos datos hay).
- `vacío` = `BUFLEN` (contador, cuántos lugares libres).

#### Protocolo correcto del productor:

```

wait(vacío)    -- esperar un lugar libre
wait(mutex)    -- entrar al buffer
poner dato
release(mutex) -- salir del buffer
release(lleno) -- avisar que hay un dato más

```

#### Protocolo correcto del consumidor:

```

wait(lleno)    -- esperar un dato disponible
wait(mutex)    -- entrar al buffer
sacar dato
release(mutex) -- salir del buffer
release(vacío) -- avisar que hay un lugar libre más

```

## Rastreo con buffer vacío (lleno=0, vacío=BUFLEN):

1. Productor hace `wait(vacío)` : vacío era BUFLEN > 0, decrementa a BUFLEN-1. Continúa.
2. Productor hace `wait(mutex)` : mutex era 1, decrementa a 0. Entra al buffer.
3. Productor pone el dato.
4. Productor hace `release(mutex)` : mutex vuelve a 1.
5. Productor hace `release(lleno)` : lleno era 0, como no hay nadie esperando, incrementa a 1.
6. Consumidor hace `wait(lleno)` : lleno era 1 > 0, decrementa a 0. Continúa.
7. Consumidor hace `wait(mutex)` : mutex era 1, decrementa a 0. Entra al buffer.
8. Consumidor saca el dato.
9. Consumidor hace `release(mutex)` : mutex vuelve a 1.
10. Consumidor hace `release(vacío)` : vacío era BUFLEN-1, incrementa a BUFLEN.

Todo funciona. Sebesta describe exactamente este patrón con `fullspots` y `emptyspots` [Sebesta, *Concepts of Programming Languages* §13.3, p. 546-547].

## El deadlock por orden invertido:

Supongamos que el productor invierte el orden de los `wait` :

```
wait(mutex)      -- PRIMERO el mutex  (ERROR)
wait(vacío)      -- después la condición
    poner dato
release(mutex)
release(lleno)
```

## Rastreo con buffer lleno (lleno=BUFLEN, vacío=0):

1. Productor hace `wait(mutex)` : mutex era 1, decrementa a 0. **Entra al buffer.**
2. Productor hace `wait(vacío)` : vacío es 0 → **se suspende en la cola de vacío**, dentro de la sección crítica.
3. Consumidor hace `wait(lleno)` : lleno era BUFLEN > 0, decrementa. Continúa.
4. Consumidor hace `wait(mutex)` : mutex es 0 (lo tiene el productor) → **se suspende en la cola de mutex.**
5. Nadie puede avanzar: el productor espera que haya lugar (que solo el consumidor puede liberar), pero el consumidor no puede entrar al buffer (porque el productor tiene el mutex).  
**Deadlock.**

**La regla:** siempre esperar la **condición de cooperación** (`vacío` para el productor, `lleno` para el consumidor) **antes** de tomar el `mutex` . Así, si te bloqueás esperando la condición, no estás dentro de la sección crítica y el otro puede entrar a producir esa condición.

Este es el punto crítico que enuncia la filmina [F-09]: “Combinar mal los `wait` puede causar deadlock.”

---

## 6. Puntos clave y resumen

---

### Las cuatro palabras que no son sinónimos

| Término      | Pregunta                                 |
|--------------|------------------------------------------|
| Concurrencia | ¿Varias actividades progresan solapadas? |
| Paralelismo  | ¿Ejecutan simultáneamente en hardware?   |
| Asincronía   | ¿Puedo continuar mientras espero?        |
| Thread       | ¿Quién ejecuta instrucciones?            |

### Las tres condiciones de carrera

1. Estado compartido mutable.
2. Al menos dos unidades lo acceden concurrentemente.
3. Al menos una lo modifica.

### Las cuatro propiedades de la sección crítica

1. Exclusión mutua.
2. Progreso.
3. Espera acotada.
4. Sin suposición de velocidad.

### Usos canónicos de los semáforos

- **Binario (contador=1):** exclusión mutua → mutex.
- **Contador (contador=N):** limitar N accesos simultáneos.
- **Cooperación:** `lleno` / `vacío` en productor/consumidor.

### Las tres partes de un monitor

1. Estado privado.
2. Procedimientos sincronizados.
3. Variables de condición.

## Síncrono vs asíncrono vs canal

| Modelo    | Quién espera      | Trade-off                          |
|-----------|-------------------|------------------------------------|
| Síncrono  | Emisor y receptor | Predecible, menos concurrencia     |
| Asíncrono | Emisor no bloquea | Más concurrencia, acumulación      |
| Canal     | Medio explícito   | Tipado y sincronización explícitos |

## Cuándo usar cada mecanismo en TypeScript

| Situación                       | Mecanismo                                             |
|---------------------------------|-------------------------------------------------------|
| Esperar I/O                     | <code>async / await</code>                            |
| Coordinar varias I/O            | <code>Promise.all</code>                              |
| Cómputo CPU-bound               | <code>worker_threads</code>                           |
| Estado compartido entre workers | <code>SharedArrayBuffer</code> + <code>Atomics</code> |

## Tres decisiones de lenguaje

| Lenguaje | Decisión                                                                            |
|----------|-------------------------------------------------------------------------------------|
| Java     | Threads del OS + <code>synchronized</code> + <code>wait / notify</code> (monitores) |
| Go       | Goroutines + channels (pasaje de mensajes)                                          |
| Kotlin   | Corrutinas + concurrencia estructurada (lifecycle)                                  |

## La idea final

El problema no es el lenguaje: es **estado compartido sin coordinación**.

## 7. Autoevaluación

Diez preguntas mezclando niveles de Bloom. Intentá responder sin mirar la guía; después abrí las respuestas al final.

**P1 (Recordar)** — Enumerá las tres condiciones que deben cumplirse simultáneamente para que haya una condición de carrera.

**P2 (Recordar)** — ¿Cuáles son las cuatro propiedades que debe garantizar toda solución correcta a la sección crítica?

**P3 (Entender)** — Explicá con tus palabras por qué un programa puede ser “concurrente sin ser paralelo”. Dale un ejemplo concreto.

**P4 (Entender)** — ¿Por qué un flag booleano simple no alcanza para garantizar exclusión mutua entre dos tareas?

**P5 (Aplicar)** — Dado el siguiente código TypeScript, decidí si hay carrera y justificá identificando cuál de las tres condiciones se cumple o no:

```
const total = 3 // ojo: const

async function A() { /* lee total, no modifica */ }
async function B() { /* lee total, no modifica */ }
A(); B();
```

**P6 (Aplicar)** — En el escenario productor/consumidor con tres semáforos ( `mutex` , `lleno` , `vacío` ), escribí el orden correcto de `wait` / `release` para el productor cuando el buffer tiene lugares libres.

**P7 (Analizar)** — Si el productor hace `wait(mutex)` antes que `wait(vacío)` y el buffer está lleno, explicá paso a paso cómo se llega al deadlock.

**P8 (Analizar)** — Compará el modelo de Java ( `synchronized` / `wait` / `notifyAll` ) con el de Go (channels). ¿Qué problema de los semáforos resuelve cada uno y de qué manera?

**P9 (Aplicar)** — Tenés que procesar 10.000 imágenes (CPU intensivo) y además consultar tres servicios externos (I/O). ¿Qué parte hacés con `Promise.all` , qué parte con `worker_threads` , y por qué no compartís el contador de imágenes procesadas sin sincronización?

**P10 (Evaluar)** — Un compañero propone usar `async` / `await` para “paralelizar” un cálculo matemático pesado que dura 10 segundos en un solo núcleo. ¿Es correcta la propuesta? Justificá refiriéndote a la distinción entre asincronía y paralelismo.

► Respuestas

---



## 8. Glosario

---

- **Concurrencia:** composición de actividades que progresan de manera solapada; no implica ejecución simultánea.
- **Paralelismo:** ejecución simultánea efectiva, usualmente sobre múltiples núcleos. Requiere hardware múltiple.
- **Asincronía:** capacidad de continuar sin bloquearse mientras se espera una operación (típicamente I/O).
- **Ejecución secuencial:** una sola secuencia de instrucciones, una tras otra, en un único hilo de control.
- **Thread (hilo):** unidad de ejecución dentro de un proceso; comparte memoria con otros threads del mismo proceso.
- **Proceso:** unidad de ejecución con espacio de memoria propio.
- **Corrutina:** unidad concurrente suspendible que el runtime mapea a threads; no es necesariamente un thread del OS.
- **Unidad concurrente:** término general para tarea, thread, proceso liviano o corrutina; lo importante es que avanza independientemente.
- **Concurrencia física:** ejecución simultánea efectiva sobre multiprocesadores o multinúcleo.
- **Concurrencia lógica (multiprogramación):** un solo núcleo intercala tareas rápidamente; el scheduler decide el orden real. Es la abstracción central de razonamiento.
- **Race condition (condición de carrera):** ocurre cuando el resultado depende del orden temporal no controlado entre accesos concurrentes a estado compartido.
- **Sección crítica:** segmento de código que accede a un recurso compartido y no debe ejecutarse concurrentemente por más de una tarea.
- **Exclusión mutua:** propiedad de que sólo una tarea puede estar dentro de la sección crítica a la vez.
- **Progreso:** propiedad de que si nadie está en la sección crítica y alguien quiere entrar, debe poder.
- **Espera acotada:** propiedad de que nadie espera indefinidamente si el recurso está libre periódicamente.
- **Semáforo:** estructura con un contador entero y una cola de espera; coordina acceso mediante `wait / release` atómicos.
- **wait (P):** operación que decrementa el contador si es mayor que cero, o suspende a la tarea en la cola si no.
- **release (V):** operación que despierta a una tarea de la cola si no está vacía, o incrementa el contador si no.
- **Semáforo binario:** semáforo con contador=1; actúa como mutex.
- **Semáforo contador:** semáforo con contador=N; limita N accesos simultáneos.
- **Monitor:** abstracción que encapsula estado compartido y procedimientos sincronizados; la entrada garantiza exclusión mutua automática.
- **Variable de condición:** mecanismo del monitor que permite a una tarea esperar dentro del monitor sin bloquear a otras.

- **Sincronización de competencia:** controla el acceso mutuamente excluyente a un recurso compartido.
  - **Sincronización de cooperación:** controla el orden relativo de ejecución cuando una unidad necesita que otra produzca una condición, dato o evento.
  - **Pasaje de mensajes:** modelo donde las unidades concurrentes coordinan intercambiando mensajes en lugar de compartir memoria.
  - **send:** primitiva que envía un mensaje a una unidad destino.
  - **receive:** primitiva que recibe un mensaje desde una unidad origen.
  - **Síncrono (mensajes):** el emisor bloquea hasta que el receptor recibe; sincronización implícita.
  - **Asíncrono (mensajes):** el emisor no bloquea; mayor concurrencia, posible acumulación.
  - **Canal:** medio explícito por donde circulan mensajes; puede ser síncrono (sin buffer) o asíncrono (con buffer); tipado en lenguajes como Go y Occam.
  - **Goroutine:** unidad de concurrencia ultra liviana de Go (~2 KB de stack); el scheduler la multiplexa sobre threads del OS.
  - **Event loop:** bucle cooperativo de JavaScript que toma tareas de la cola y las ejecuta hasta completion; habilita concurrencia de espera.
  - **Worker (worker\_threads / Web Worker):** unidad separada que ejecuta código en paralelo; sirve para cómputo CPU-bound; se comunica por mensajes.
  - **SharedArrayBuffer:** bloque de memoria compartida entre workers; permite compartir estado pero reintroduce problemas de sincronización.
  - **Atomics:** API de operaciones atómicas sobre `SharedArrayBuffer` ; base de bajo nivel para implementar mutexes y semáforos.
  - **Promise.all:** primitiva de TypeScript/JavaScript que coordina varias promesas y espera todas; concurrencia de espera, no paralelismo.
  - **async/await:** sintaxis que expresa suspensión sobre el event loop; no crea paralelismo de CPU.
  - **Runnable:** interfaz de Java que separa la tarea (lógica) del mecanismo ( `Thread` ); un objeto puede implementar `Runnable` y extender otra clase.
  - **Deadlock:** situación donde dos o más unidades se bloquean mutuamente esperando recursos que ninguna puede liberar.
  - **Concurrencia estructurada:** modelo donde las unidades concurrentes viven dentro de un scope que gobierna ciclo de vida, cancelación y errores (Kotlin).
-

## 9. Referencias y lecturas recomendadas

**Bibliografía primaria (verificada en ChromaDB, colección `edu_knowledge` , tipo `material` )**

- **Robert W. Sebesta**, *Concepts of Programming Languages*, Pearson, 2019, **Cap. 13: Concurrency**.
  - §13.1 Introduction (niveles de concurrencia, p. 537).
  - §13.2 Fundamental Concepts: tarea, race condition (“two or more tasks are racing to use the shared resource”, p. 539), competencia y cooperación (p. 539-540).
  - §13.3 Semaphores: contador + cola, wait/release, productor/consumidor con `fullspots` / `emptyspots` (p. 544-547).
  - §13.4 Monitores: shared data residente en el monitor (p. 565).
  - §13.5 Message Passing: synchronous message passing (p. 544+).
  - §13.6 Java: `Thread` / `Runnable` , `wait` / `notify` / `notifyAll` (p. 565-566).
- **Maurizio Gabbrielli y Simone Martini**, *Programming Languages: Principles and Paradigms*, Springer, 2023, **Cap. 14: Concurrent Programming** (pp. 447-486).
  - Semáforos introducidos por Dijkstra en los 60s.
  - Monitores: exclusión mutua implícita, variables de condición para sincronización de condición.
  - Canales: “first appeared at the programming-language level in Occam” (p. 453).
  - Comunicación síncrona vs asíncrona y su equivalencia expresiva.
  - Java: `synchronized` da exclusión mutua pero no sincronización de condición; `wait` / `notify` / `notifyAll` para cooperación.
- **Kenneth C. Louden y Kenneth A. Lambert**, *Programming Languages: Principles and Practices*, Course Technology, 2012, **Cap. 13: Concurrency / Parallel Programming** (pp. 585-666).
  - `wait` / `sleep` como operaciones de suspensión; `notify` / `notifyAll` como operaciones de continuación (p. 610).
  - Productor/consumidor en Java con `notifyAll` (p. 604).
  - Apoyo terminológico sobre unidades concurrentes y coordinación.

### Documentación oficial (contrastess de lenguaje)

- **MDN Web Docs — JavaScript execution model:** [https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Execution\\_model](https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Execution_model)
- **Node.js Documentation — worker\_threads:** [https://nodejs.org/api/worker\\_threads.html](https://nodejs.org/api/worker_threads.html)
- **Kotlin Documentation — Coroutines overview:** <https://kotlinlang.org/docs/coroutines-overview.html>
- **Go Documentation — Memory Model:** <https://go.dev/ref/mem>

## Referencia histórica

- **Ada** aparece como referencia histórica de tareas y *rendezvous* (pasaje de mensajes síncrono). No se desarrolla en esta guía por estar fuera del scope de la clase.

---

**Nota de trazabilidad:** todas las citas bibliográficas de esta guía ( [Sebesta, ...] , [Gabbrielli & Martini, ...] , [Louden & Lambert, ...] ) fueron verificadas con `python scripts/knowledge_base.py search "..."` --type material contra la base ChromaDB local (colección `edu_knowledge` ). Las páginas citadas corresponden a los hits devueltos por ChromaDB. No se inventaron citas ni páginas. Las URLs de documentación oficial provienen de `.pipeline-v3-state.yaml` y `topic-extract.md` .